

Reporte Tarea 8

Jesús Galaviz Tapia

En este trabajo se resolvió numéricamente el problema de Thomson usando Python: se implementaron los métodos de **Metropolis-Montecarlo** y el de **descenso de gradiente** propuesto por Claxton y Benson con el fin de calcular la energía electrostática (en unidades de $1/4\pi\epsilon_0$) y las configuraciones correspondientes de N partículas con carga $+1$ distribuidas sobre una esfera unitaria. Siendo r_{ij} la distancia Euclidiana entre las partículas i y j , dicha energía está dada por

$$U_T = \frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{ij}},$$

El archivo de salida especifica las configuraciones de las partículas obtenidas y su correspondiente energía electrostática, en formato **x,y,z**.

En este trabajo se emplearon las librerías NumPy y Numba. La librería Numba se utilizó para compilar el código a un backend en C, evitando así la ejecución mediante el intérprete de Python y por lo tanto disminuyendo el tiempo de cómputo. Numba sólo puede optimizar funciones que operan con estructuras y métodos de NumPy (por ejemplo al definir la matriz que contiene las posiciones de las partículas o al generar un número aleatorio), por ello fue indispensable el uso de esta última.

Puesto que se usa el algoritmo de *Metrópolis*, se definen temperaturas de trabajo $T_k = k\delta T$ en las cuales se realizan ciclos Montecarlo. δT es el intervalo que existe entre una temperatura de trabajo y la siguiente.

Parámetros

A lo largo de la simulación se usaron los siguientes parámetros, posteriormente se presentan los métodos usados. Se indica en qué métodos se usó cada parámetro.

- N : número de partículas puntuales (métodos 1,2,3).
- m : número de temperaturas de trabajo T_k (método 1).
- T : temperatura máxima de trabajo. En la simulación se usó $T = 10^{-4}$ (método 1).
- n : número de ciclos montecarlo por temperatura de trabajo (método 1).
- L : longitud de la caja de desplazamiento (método 1).
- w : número total de iteraciones del algoritmo Claxton–Benson (métodos 2,3).
- K : parámetro de control que regula la magnitud del desplazamiento de las partículas en cada iteración. En este caso, se usó $K = 0.15$ (métodos 2,3).

Metrópolis-Montecarlo (método 1)

Metodología

El método empleado corresponde a una implementación del algoritmo de *Metrópolis* para minimizar la energía electrostática del conjunto de partículas puntuales sobre una esfera. Puesto que hay m temperaturas de trabajo igualmente espaciadas con espaciamiento δT , y la máxima es T , entonces $\delta T = T/m$, y por lo tanto $T_k = Tk/m$.

El problema de Thomson no contempla temperatura, por lo tanto debemos disminuir T_k desde su valor máximo hasta el valor mínimo T/m que debe ser muy cercano a cero para que en las proximidades de esta temperatura el factor de Boltzmann sea 0, y por lo tanto sólo se acepten movimientos que minimicen la la energía; haciendo esto se garantiza que la última configuración obtenida tenga una energía electrostática menor al de las anteriores (dentro de aquellas en el conjunto de temperaturas T_k finales), y puesto que aceptamos configuraciones no favorables con cierta probabilidad, se exploran más configuraciones (pero con energías electrostáticas muy cercanas).

1. **Inicialización de coordenadas.** Se genera un conjunto inicial de N posiciones aleatorias sobre la superficie de la esfera unitaria. Para cada partícula i , se eligen tres coordenadas aleatorias (x_1, x_2, x_3) en el intervalo $[-1, 1]$ y se normaliza el vector resultante:

$$\vec{r}_i = \frac{(x_1, x_2, x_3)}{\sqrt{x_1^2 + x_2^2 + x_3^2}},$$

asegurando que todas las partículas se ubiquen en la superficie de la esfera de radio unitario.

2. **Cálculo de energía.** La energía total del sistema se calcula mediante la función `f_u(crd)`, que realiza una doble suma sobre todos los pares (i, j) , evaluando la interacción electrostática U_T .

Para calcular la energía parcial asociada a una sola partícula k , se utiliza la función `f_eng(x1, x2, x3, k, crd)`, que evalúa la contribución de dicha partícula con todas las demás. Es decir, calcula

$$U = \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}}.$$

3. **Evolución Metópolis-MonteCarlo.** En cada ciclo Metópolis-MonteCarlo, implementado en la función `MCS(crd, N, m)`, se realizan los siguientes pasos:

- Se selecciona una partícula al azar.
- Se genera una posición *trial* desplazando aleatoriamente sus coordenadas.
- La posición *trial* se normaliza nuevamente para mantener la partícula sobre la esfera.

- Se evalúa la energía electrostática de la partícula en la posición propuesta y en la posición actual. Si

$$U_{\text{trial}} - U_{\text{actual}} = \Delta U \leq 0,$$

la nueva posición es aceptada; esto es equivalente a que $U'_T \leq U_T$, es decir, que la energía total del sistema cuando cambiamos la partícula k por la posición *trial* es menor que la correspondiente a la posición anterior. Esto se demuestra en el apéndice.

Suponiendo una temperatura de trabajo T_k y en caso de que $\Delta U > 0$, la posición *trial* se acepta con una probabilidad $P = \exp(-\Delta U/T_k)$: se genera un número aleatorio con densidad de probabilidad uniforme η entre 0 y 1, si $\eta < P$ entonces se acepta el movimiento.

4. **Obtención de resultados.** Tras completar los $n \times m$ ciclos, se calcula la energía total del sistema mediante `f_u(crd)`. Las coordenadas finales se guardan en el archivo de texto `coord_Metro_N.txt` (N es el número de partículas) junto con el valor final de la energía electrostática. Finalmente, se compara el resultado obtenido con el valor exacto correspondiente a N para determinar el error porcentual.

Claxton-Benson (método 2)

Metodología

El principio fundamental del método consiste en mover las partículas en la dirección de la fuerza electrostática resultante que actúa sobre cada una, pero con una magnitud controlada por un factor adaptativo γ , de modo que todas permanezcan sobre la superficie de la esfera. Contiene los siguientes pasos:

1. **Inicialización de las posiciones.** Se generan N posiciones aleatorias sobre la esfera unitaria utilizando la función `f_coord(N)`. Cada partícula se obtiene normalizando un vector aleatorio tridimensional (x_1, x_2, x_3) cuyos componentes se distribuyen uniformemente en $[-1, 1]$:

$$\vec{r}_i = \frac{(x_1, x_2, x_3)}{\sqrt{x_1^2 + x_2^2 + x_3^2}}.$$

De esta manera, se garantiza que $|\vec{r}_i| = 1$ para toda partícula.

2. **Cálculo de la energía y de las fuerzas.** La energía electrostática total se calcula mediante la función `f_u(crd)`. Por otro lado, la fuerza resultante que siente la partícula k se calcula en la función `F_vect(k, crd)` como:

$$\vec{F}_k = \sum_{i \neq k} \frac{\vec{r}_k - \vec{r}_i}{|\vec{r}_k - \vec{r}_i|^3},$$

que corresponde a la expresión habitual de la fuerza electrostática (en unidades reducidas) debida a todas las demás partículas.

3. **Definición del parámetro de paso γ .** En cada iteración del método, el parámetro γ se calcula de forma adaptativa según:

$$\gamma = \frac{K}{F_{\text{máx}}},$$

donde $F_{\text{máx}}$ es el módulo de la fuerza más intensa entre todas las partículas en la configuración actual. Este valor se determina mediante la función `F_max(crd)`. Esta definición asegura que el desplazamiento máximo de cualquier partícula sea controlado y que el sistema no diverja numéricamente durante la evolución.

4. **Iteración Claxton–Benson.** El núcleo del método se encuentra en la función `CB(crd)`, la cual realiza w iteraciones. En cada iteración:

- Se calcula la fuerza $\vec{F}_j$ sobre cada partícula j .
- Se actualiza su posición siguiendo:

$$\vec{r}_j = \frac{\vec{r}_j + \gamma \vec{F}_j}{|\vec{r}_j + \gamma \vec{F}_j|}.$$

La normalización garantiza que la partícula permanezca sobre la esfera unitaria.

Este proceso desplaza todas las partículas de forma simultánea en la dirección de la fuerza neta que experimentan, pero con un paso reducido proporcional a $1/F_{\text{máx}}$.

5. **Criterio de convergencia y resultados.** Al finalizar, se calcula la energía total U_{obt} y se compara con el valor exacto del problema de Thomson para el mismo número de partículas. Las posiciones finales de las partículas, junto con la energía, se guardan en un archivo de texto `coord_C_N.txt` (N es el número de partículas especificado).

Claxton-Benson y Metrópolis-Montecarlo (método 3)

Considerando N partículas, el tercer método implementado consiste en tomar la configuración generada por el método 1 (*Metrópolis-Montecarlo*) y aplicarle a esta el método 2 (*Claxton-Benson*). Las posiciones finales de las partículas, junto con la energía, se guardan en el archivo de texto llamado `coord_Metro-C_N.txt` (N es el número de partículas especificado).

A continuación se presentan los resultados obtenidos por los tres métodos, además se enuncian algunas conclusiones.

Resultados y Conclusiones

Se muestran las energías obtenidas para cada uno de los métodos usados, mostrando buena relación con los resultados exactos. Es importante mencionar que en lo que sigue se buscaron parámetros óptimos, tales que su tamaño no implicara inestabilidad numérica. Se tienen las siguientes conclusiones:

- el método 2 produce menos error comparado con los demás para $N \leq 1000$. Para $N = 10000$ el método 1 produce el mejor resultado
- el método 3 mejora significativamente los resultados del método 1 para todos los valores de N , excepto para $N = 10000$
- debido a la precisión numérica de una computadora, usar valores muy chicos de T resulta en errores más grandes de resultados respecto a aquellos exactos, en este caso se notó que $T = 1 \times 10^{-4}$ es un valor adecuado
- a medida que N aumenta (alrededor de 1000), son necesarios menos ciclos en ambos métodos. Esto se puede entender debido a que las configuraciones para N grande son cada vez más cercanas a la del equilibrio: las fuerzas resultantes se vuelven mayores, y en el esquema de Claxton y Benson las posiciones casi no se actualizan, ya que γ en estos casos es cercano a 0. De la misma manera, para el método 1, al haber muchas partículas, es poco probable que se generen movimientos significativos en cada intento de Monte Carlo, especialmente para T_k cercanas a δT

Las siguientes tablas muestran las energías obtenidas para los distintos números de partículas considerados, $N = 2, 3, 4, 10, 20, 40, 80, 100, 1000, 10000$, empleando los tres métodos; además se compara con los valores proporcionados en [2]. Por último se especifican los detalles relevantes del hardware usado, así como la configuración resultante para $N = 1000$ obtenida con el método 2.

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.50000008	0.5	1.7×10^{-5}	A
3	1.732051	1.732050	5.9×10^{-5}	A
4	3.674236	3.674234	6.4×10^{-5}	A
10	32.7170	32.7169	0.0001	A
20	150.8830	150.8815	0.0009	A
40	660.7740	660.6752	0.0149	A
80	2805.7525	2805.3558	0.0141	A
100	4449.1406	4448.3506	0.0177	A
1,000	482630.1	482535.0	0.0197	A
10,000	49467584.5	49447400.0	0.0408	A

Cuadro 1: Resultados obtenidos usando el método 1

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.5	0.5	0.0	A
3	1.732050807	1.732050807	0.0	A
4	3.674234614	3.674234614	0.0	A
10	32.71694946	32.7169494	1.8×10^{-7}	A
20	50.88156833	150.8815683	2.2×10^{-8}	A
40	660.67527883	660.6752788	5.2×10^{-9}	A
80	2805.35587598	2805.3558759	2.8×10^{-9}	A
100	4448.35066	4448.35063	7.7×10^{-7}	A
1,000	482536.1	482535.0	0.0002	A
10,000	49825446.5	49447400.0	0.7645	C

Cuadro 2: Resultados obtenidos usando el método 2

N	Energía obtenida	Resultado exacto	Error porcentual	Calificación
2	0.5	0.5	0.0	A
3	1.732050807	1.732050807	0.0	A
4	3.674234614	3.674234614	0.0	A
10	32.716949	32.7169494	1.8×10^{-7}	A
20	150.88156833	150.8815683	2.2×10^{-8}	A
40	660.725	660.7253	0.0075	A
80	2805.43	2805.35	0.0029	A
100	4448.35063433	4448.3506343	7×10^{-10}	A
1,000	482535.9	482535.0	0.0001	A
10,000	49844030.4	49447400.0	0.8021	C

Cuadro 3: Resultados obtenidos usando el método 3

Se presentan los tiempos tomados por la simulación 1, asociados a los parámetros usados, realizada en una PC. De la misma manera se presentan aquellos cuando se realiza en Colab.

N	Número de ciclos MC	Tiempo de cómputo (s)	m	n	L
2	180,000	0.2636	300	600	.5
3	180,000	0.2791	300	600	.5
4	180,000	0.3596	300	600	.5
10	180,000	0.4873	300	600	.5
20	120,000	0.6541	200	600	.5
40	100,000	1.2580	200	500	.5
80	50,000	1.9857	100	500	.5
100	50,000	2.8890	100	500	.5
1,000	10,000	44.8749	20	500	.5
10,000	500	220.5472	5	100	.5

Cuadro 4: Tiempos tomados para el método 1, para distintos números de partículas N , usando los parámetros indicados. Correspondientes a una PC

N	Número de ciclos MC	Tiempo de cómputo (s)	m	n	L
2	180,000	0.4112	300	600	.5
3	180,000	0.4354	300	600	.5
4	180,000	0.5610	300	600	.5
10	180,000	0.7602	300	600	.5
20	120,000	1.0204	200	600	.5
40	100,000	1.9625	200	500	.5
80	50,000	3.0987	100	500	.5
100	50,000	4.5108	100	500	.5
1,000	10,000	70.0088	20	500	.5
10,000	500	344.0536	5	100	.5

Cuadro 5: Tiempos tomados para el método 1, para distintos números de partículas N , usando los parámetros indicados. Esta vez usando Colab

El **dispositivo usado** fué una Mac M1 del 2020, cuyas características son:

- cpu de 8 núcleos, arquitectura ARMv8-A de 64 bits
- 4 de los núcleos de 3.2 Ghz, los otros 4 de 2.1 Ghz
- caches asociados de 12 Mb y 4 Mb respectivamente
- ancho de banda de memoria de 68.25 GB/s
- Memoria unificada de 8 GB LPDDR4X a 4266 MHz, compartida entre CPU y GPU, con arquitectura de acceso unificado.

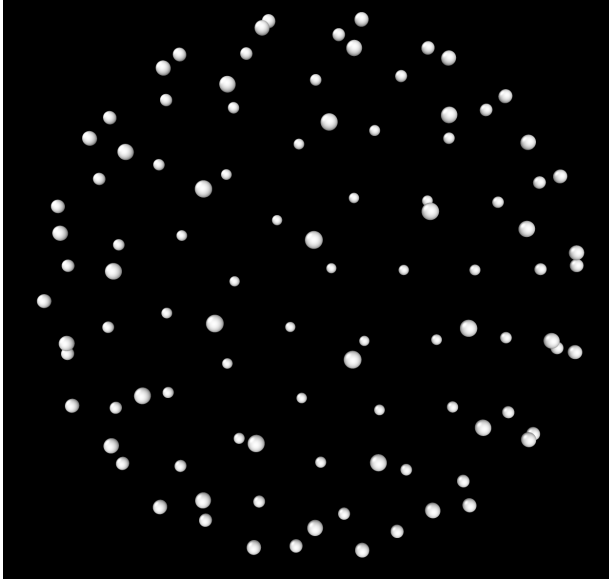


Figura 1: Configuración obtenida mediante el método 2, para $N = 100$

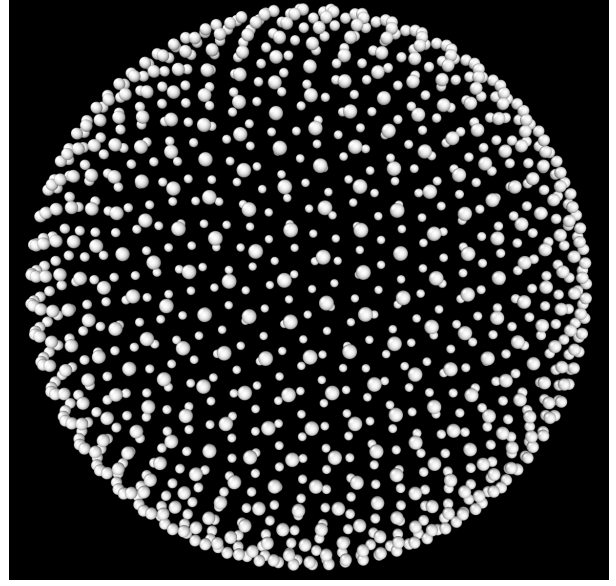


Figura 2: Configuración obtenida mediante el método 2, para $N = 1000$

Demostración

La energía electrostática del conjunto de las N partículas es

$$U_T = \frac{1}{2} \sum_{i=1}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{i,j}},$$

donde $r_{i,j}$ es la distancia entre las partículas i y j . Fijándonos en una partícula arbitraria, la k -ésima, lo anterior se puede reescribir como

$$U_T = \frac{1}{2} \sum_{\substack{i=1 \\ i \neq k}}^N \sum_{\substack{j=1 \\ j \neq i}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

Intercambiando el orden de sumación en el primer término del lado derecho de la ecuación anterior, se obtiene

$$U_T = \frac{1}{2} \sum_{j=1}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

De la misma manera que al principio, fijándonos en la partícula k , podemos reescribir la energía electrostática como

$$U_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \frac{1}{2} \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}} + \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \frac{1}{r_{k,j}}.$$

Debido a que $r_{i,j}$ es simétrico en los índices, lo anterior es igual a

$$U_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r_{i,k}}.$$

Nótese que no existe dependencia en el primer término del lado derecho de la ecuación respecto de la posición de la partícula k . Si $r'_{i,k}$ representa la distancia entre la partícula i y la partícula k al haber cambiado esta última de posición, entonces se obtiene una nueva energía:

$$U'_T = \frac{1}{2} \sum_{\substack{j=1 \\ j \neq k}}^N \sum_{\substack{i=1 \\ i \neq k \\ i \neq j}}^N \frac{1}{r_{i,j}} + \sum_{\substack{i=1 \\ i \neq k}}^N \frac{1}{r'_{i,k}}.$$

Si se restan ambas ecuaciones se obtiene

$$U'_T - U_T = \sum_{\substack{i=1 \\ i \neq k}}^N \left(\frac{1}{r'_{i,k}} - \frac{1}{r_{i,k}} \right) = \Delta U,$$

de manera que

$$U'_T = \Delta U + U_T.$$

Referencias

- [1] O. Steinhauser, *Computer Simulation in Physics and Engineering*, De Gruyter, 2013.
- [2] D. J. Wales y S. Ulker, “Global Minima for the Thomson Problem,” *The Cambridge Cluster Database*, disponible en: <https://www-wales.ch.cam.ac.uk/~wales/CCD/Thomson/table.html>